

Socket

■ Gestione interruzione

- send() o recv() possono essere interrotte
- EINTR vale per tutte le syscall bloccanti (interrotte da segnale ritornano -1 e EINTR)

■ Perché su read/write su file non gestite:

- Molte libc ri-eseguono la read direttamente
- Su socket, pipe, sleep, accept, etc. va gestita

■ Con SA_RESTART

- Syscall bloccante interrotta da segnale viene rieseguita invece di -1 ed EINTR
- Altrimenti va gestita a mano

```
while (read(...) < 0 && errno == EINTR) continue;
```

■ Non sempre SA_RESTART va bene

- select(), waited(), nanosleep() non funziona correttamente

Socket

■ Gestione interruzione

- send() o recv() possono essere interrotte
- EINTR vale per tutte le syscall bloccanti (interrotte da segnale ritornano -1 e EINTR)

■ Gestione di SA_RESTART

- Solo sigaction() permette di impostare SA_RESTART

```
struct sigaction {  
    void (*sa_handler)(int);  
    sigset_t sa_mask;  
    int sa_flags; // <— contiene SA_RESTART  
};
```

```
struct sigaction sa = {0};  
sa.sa_handler = mio_handler;  
sa.sa_flags = SA_RESTART;  
sigaction(SIGUSR1, &sa, NULL);
```

Socket

■ Gestione interruzione

- send() o recv() possono essere interrotte
- EINTR vale per tutte le syscall bloccanti (interrotte da segnale ritornano -1 e EINTR)

■ Gestione di accept senza SA_RESTART

- Abbiamo: accept() → segnale → handler → ritorno → accept() fallisce
- → -1, errno = EINTR
- Quindi impostata manualmente:

```
for (;;) {  
    int sd = accept(listen_sd, ...);  
    if (sd >= 0) break;  
    if (errno == EINTR) continue;  
    perror("accept");  
}
```

Socket

■ Gestione interruzione

- send() o recv() possono essere interrotte
- EINTR vale per tutte le syscall bloccanti (interrotte da segnale ritornano -1 e EINTR)

■ Gestione di accept con SA_RESTART

- Abbiamo: accept() → segnale → handler → ritorno → accept() viene riavviata
- Quindi in Linux riavviata in automatico, ma non POSIX
- Impostare:

```
struct sigaction sa;  
memset(&sa, 0, sizeof(sa));  
sa.sa_handler = mio_handler;  
sa.sa_flags = SA_RESTART;  
sigemptyset(&sa.sa_mask);  
  
sigaction(SIGINT, &sa, NULL);
```

Socket

■ Gestione SIGPIPE

- Quando viene chiusa una socket chi scrive riceve un SIGPIPE
- Si può gestire ignorando + errno
- Oppure si può gestire con handler di segna,e

■ Metodo:

```
signal(SIGPIPE, SIG_IGN);
```

```
ssize_t w = send(sd, buf, len, 0);  
if (w < 0) {  
    if (errno == EPIPE) {  
        // peer chiuso  
        close(sd);  
    }  
}
```

■ Comportamento:

- In questo caso ignorato
- Si gestisce con errno nel caso di write/signal
- Nel caso di read/send, se legge 0 byte la connessione è interrotta

Socket

■ Gestione SIGPIPE

- Quando viene chiusa una socket chi scrive riceve un SIGPIPE
- Si può gestire ignorando + errno
- Oppure si può gestire con handler di segna,e

■ Metodo:

```
signal(SIGPIPE, SIG_IGN);
```

```
ssize_t n = recv(sd, buf, sizeof(buf), 0);  
if (n == 0) {  
    // Il peer ha chiuso la connessione (FIN)  
    printf("peer ha chiuso la connessione\n");  
    close();  
}
```

■ Comportamento:

- In questo caso ignorato
- Si gestisce con errno nel caso di write/signal
- Nel caso di read/send, se legge 0 byte la connessione è interrotta

Socket

■ Gestione SIGPIPE

- Quando viene chiusa una socket chi scrive riceve un SIGPIPE
- Si può gestire ignorando + errno
- Oppure si può gestire con handler di segnale

■ Metodo:

```
if (pthread_key_create(&key_sd, NULL) != 0) {  
    perror("pthread_key_create");  
    return 1;  
}
```

```
if (signal(SIGPIPE, sigpipe_handler) == SIG_ERR) {  
    perror("signal");  
    return 1;  
}
```

■ Comportamento:

- In questo caso ogni thread conserva la sua chiave privata
- Quando scatta il segnale, il gestore prende la chiave e chiude
- Attenzione a doppie chiusure (comportamento non definito)

Socket

■ Gestione SIGPIPE

- Quando viene chiusa una socket chi scrive riceve un SIGPIPE
- Si può gestire ignorando + errno
- Oppure si può gestire con handler di segnale

■ Metodo:

```
static void sigpipe_handler(int signo)
{
    (void)signo;

    // Recupera il socket associato al thread attuale
    int sd = (int)(intptr_t)pthread_getspecific(key_sd);
    if (sd >= 0) {
        close(sd);
        // Evito doppi close, usa int → intptr_t → void*
        pthread_setspecific(key_sd, (void*)(intptr_t)-1);
    }

    const char msg[] = "SIGPIPE: chiuso socket thread\n";
    write(STDERR_FILENO, msg, sizeof(msg)-1);
}
```


Select

- `select()` permette di:
 - Attendere evento I/O su più socket contemporaneamente senza bloccarne uno mentre gli altri hanno dati pronti
 - Permette di evitare un thread per socket
 - Utile per gestire molti client in modo single-threaded

Select

```
int select (int numfds, fd_set *readfds, fd_set *writefds, fd_set *exceptfds,  
            struct timeval *timeout);
```

Header file sys/time.h, sys/types.h, unistd.h

- Controlla contemporaneamente lo stato di uno o più descrittori specificati
- Si blocca finché:
 - non avviene un'attività (lettura/scrittura) su un descrittore degli insiemi
 - non viene generata un'eccezione
 - non scade un timeout
- Restituisce
 - -1 in caso di errore
 - 0 se il timeout è scaduto
 - Altrimenti, il numero totale di descrittori

Parametri Select

Insiemi di descrittori da controllare

- **readfds**: pronti per operazioni di lettura

 - Es: socket pronto per la lettura se c'è una connessione in attesa di accept()

- **writefds**: pronti per operazioni di scrittura

- **exceptfds**: exceptional conditions, quasi sempre nullo

readfds, writefds e exceptfds sono puntatori a variabili di tipo fd_set

- fd_set insieme dei descrittori (è una bit mask implementata con un array di interi)

numfds è il numero/range massimo di descrittori controllati da select()

- **ATTENZIONE**: Se maxd è il massimo descrittore usato, numfds = maxd + 1

- Può essere posto uguale alla costante FD_SETSIZE

Timeout

- timeout specifica il valore massimo che la funzione select() attende per individuare un descrittore pronto

```
struct timeval {  
    int tv_sec;      // seconds  
    int tv_usec;    // microseconds  
};
```

- Se impostato a NULL (timeout == NULL)
 - select si blocca indefinitamente fino a quando è pronto un descrittore
- Se impostato a zero (timeout->tv_sec == 0 && timeout->tv_usec == 0)
 - select non attende (da usare per polling dei descrittori senza bloccare)
- Se diverso da zero (timeout->tv_sec != 0 || timeout->tv_usec != 0),
 - select attende il tempo specificato
 - select() ritorna, se è pronto, uno (o più) dei descrittori specificati (numero positivo) oppure 0 se è scaduto il timeout

Impostazione degli insiemi

- Macro utilizzate per gestire gli insiemi di descrittori

Function	Description
<code>FD_SET(int fd, fd_set *set);</code>	Add <code>fd</code> to the set.
<code>FD_CLR(int fd, fd_set *set);</code>	Remove <code>fd</code> from the set.
<code>FD_ISSET(int fd, fd_set *set);</code>	Return true if <code>fd</code> is in the set.
<code>FD_ZERO(fd_set *set);</code>	Clear all entries from the set.

`int FD_ISSET(int fd, fd_set *set);`

- Al ritorno di `select()`, controlla se `fd` appartiene all'insieme di descrittori `set`, verificando se il bit relativo a `fd` è pari a 1 (restituisce 0 in caso negativo, un valore diverso da 0 in caso affermativo)

Select

La funzione `select()` rileva i descrittori pronti (non bloccati)

- Significato diverso per i tre tipi di descrittori (lettura, scrittura, exceptional conditions)
- Un descrittore è **pronto in lettura** se:
 - Nel buffer di ricezione del socket sono arrivati byte in quantità sufficiente (soglia minima per default pari a 1, modificabile con opzione del socket `SO_RCVLOWAT`)
 - Il peer ha chiuso il suo lato di scrittura (ricevuto FIN, leggerà fino ad EOF)
 - Si è verificato un errore sul socket
 - Se è un socket di ascolto (LISTEN) e ci sono delle connessioni pendenti
- Un descrittore è **pronto in scrittura** se:
 - Nel buffer di invio del socket c'è spazio sufficiente per inviare (soglia minima per default pari a 2048 byte, modificabile con opzione del socket `SO_SNDLOWAT`) ed il socket è già connesso (TCP) oppure non necessita di connessione (UDP)
 - La connessione TCP è stabilita (per UDP sempre ok)
 - Il peer ha chiuso il suo lato di lettura
 - C'è errore pendente sul socket

Select

La funzione `select()` rileva i descrittori pronti (non bloccati)

- Significato diverso per i tre tipi di descrittori (lettura, scrittura, eccezione)
- **Exceptional conditions:**
 - In Linux, l'unica condizione per cui `select()` segnala fd in `exceptfds` è OOB TCP data
 - Out-of-band data (OOB) per TCP → SIGURG / MSG_OOB
 - Dati urgenti segnalati così
 - Di leggono con: `recv(fd, buf, len, MSG_OOB);`

```
if (FD_ISSET(c, &efds)) {  
    char buf[1];  
    printf(">>> OOB READY!\n");  
    int r = recv(c, buf, sizeof(buf),  
MSG_OOB);  
    if (r > 0) {  
        printf("OOB DATA: %c\n", buf[0]);  
    } else { perror("recv OOB"); }  
}
```

Esempio

```
#include <stdio.h>
#include <sys/time.h>
#include <sys/types.h>
#include <unistd.h>
#include <sys/select.h>
#define STDIN 0 // file descriptor for standard input

int main(void)
{
    struct timeval tv;
    fd_set readfds;

    tv.tv_sec = 2;
    tv.tv_usec = 500000;

    FD_ZERO(&readfds);
    FD_SET(STDIN, &readfds);

    // don't care about writefds and exceptfds:
    select(STDIN+1, &readfds, NULL, NULL, &tv);

    if (FD_ISSET(STDIN, &readfds))
        printf("A key was pressed!\n");
    else
        printf("Timed out.\n");

    return 0;
}
```


Esempio

```
#include <stdio.h>
#include <unistd.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/select.h>
#include <sys/time.h>
```

```
int main(void) {
    fd_set readfds;
    struct timeval tv;
    int nfd = STDIN_FILENO + 1;

    FD_ZERO(&readfds);
    FD_SET(STDIN_FILENO, &readfds);

    tv.tv_sec = 2;
    tv.tv_usec = 500000; // 2.5 s (usec: milionesimi di sec)

    int n = select(nfd, &readfds, NULL, NULL, &tv);
    if (n < 0) {
        if (errno == EINTR) { /* ripetere se necessario */ }
        perror("select"); return 1;
    } else if (n == 0) { printf("Timed out.\n"); }
    else if (FD_ISSET(STDIN_FILENO, &readfds)) {
        printf("Input disponibile (tipicamente dopo Invio).\n");
        char buf[128]; // Esempio: leggere qualcosa
        ssize_t r = read(STDIN_FILENO, buf, sizeof buf);
        if (r > 0) write(STDOUT_FILENO, buf, (size_t)r);
    }
    return 0;
}
```

Multiplexing Server

Il multiplexing dell'I/O può essere usato sul server per ascoltare su più socket contemporaneamente

- Un unico processo server (iterativo) ascolta sul socket di ascolto e sui socket di connessione

Struttura generale di un server che usa `select()`

- riempie una struttura `fd_set` con i descrittori dai quali si intende leggere
- riempie una struttura `fd_set` con i descrittori sui quali si intende scrivere
- chiama `select()` ed attendere finché non avviene qualcosa
- quando `select()` ritorna, controlla se uno dei descrittori ha causato il ritorno. In questo caso, serve il descrittore in base al tipo di servizio offerto
- ripetere il ciclo

Multiplexing Client

Il client deve controllare due diversi descrittori in lettura

- Lo standard input, da cui legge il testo da inviare al server
- Il socket connesso con il server:
 - su cui scriverà il testo e dal quale riceverà la risposta

Multiplexing Client

Il client deve controllare due diversi descrittori in lettura

Si ricrea il set ad ogni ciclo perché **select() consuma: trasforma il set nell'insieme degli fd pronti**

```
void str_cli_echo_sel(FILE *fd, int sockfd)
{
    int                maxd, n;
    fd_set             rset;
    char               sendline[MAXLINE], recvline[MAXLINE];

    FD_ZERO(&rset);          /* inizializza a 0 il set dei descrittori in lettura */
    for ( ; ; ) {
        FD_SET(fileno(fd), &rset); /* inserisce il descrittore del file (stdin) */
        FD_SET(sockfd, &rset);    /* inserisce il descrittore del socket */
        maxd = (fileno(fd) < sockfd) ? (sockfd + 1) : (fileno(fd) + 1);
        if (select(maxd, &rset, NULL, NULL, NULL) < 0 ) { /* attende descrittore pronto
            in lettura */
            perror("errore in select");
            exit(1);
        }
    }
}
```

Multiplexing Client

Il client deve controllare due diversi descrittori in lettura

```
/* Controlla se il file (stdin) è leggibile */
if (FD_ISSET(fileno(fd), &rset)) {
    if (fgets(sendline, MAXLINE, fd) == NULL)
        return; /* non vi sono dati perché si è concluso l'utilizzo del client */
    if ((writen(sockfd, sendline, strlen(sendline))) < 0) {
        fprintf(stderr, "errore in write");
        exit(1);
    }
}
```

Multiplexing Client

Il client deve controllare due diversi descrittori in lettura

```
/* Controlla se il socket è leggibile */
if (FD_ISSET(sockfd, &rset)) {
    if ((n = readline(sockfd, recvline, MAXLINE)) < 0) {
        fprintf(stderr, "errore in lettura");
        exit(1);
    }
    if (n == 0) {
        fprintf(stdout, "str_cli_echo_sel: il server ha chiuso la connessione");
        return;
    }
    /* Stampa su stdout */
    recvline[n] = 0;
    if (fputs(recvline, stdout) == EOF) {
        perror("errore in scrittura su stdout");
        exit(1);
    }
}
}
```

Multiplexing Server

Echo-server serve più connessioni contemporaneamente, con un solo thread, per questo c'è l'array `client[]`. Mantiene 2 insiemi di fd: `allset` e `rset`

```
/* L'array di interi client contiene i descrittori dei socket connessi */
for (i = 0; i < FD_SETSIZE; i++)
    client[i] = -1;

FD_ZERO(&allset); /* Inizializza a zero l'insieme dei descrittori */
FD_SET(listensd, &allset); /* Inserisce il descrittore di ascolto */

for ( ; ; ) {
    rset = allset; /* Imposta il set di descrittori per la lettura */
    /* ready è il numero di descrittori pronti */
    if ((ready = select(maxd+1, &rset, NULL, NULL, NULL)) < 0) {
        perror("errore in select");
        exit(1);
    }
    /* Se è arrivata una richiesta di connessione, il socket di ascolto
       è leggibile: viene invocata accept() e creato un socket di connessione */
    if (FD_ISSET(listensd, &rset)) {
        len = sizeof(cliaddr);
```


Multiplexing Server

```
if ((connsd = accept(listensd, (struct sockaddr *)&cliaddr, &len)) < 0) {
    perror("errore in accept");
    exit(1);
}

/* Inserisce il descrittore del nuovo socket nel primo posto libero di client */
for (i=0; i<FD_SETSIZE; i++)
    if (client[i] < 0) {
        client[i] = connsd;
        break;
    }
/* Se non ci sono posti liberi in client, errore */
if (i == FD_SETSIZE) {
    fprintf(stderr, "errore in accept");
    exit(1);
}
```


Multiplexing Server

ready = select(...): numero di descrittori pronti; dopo ogni gestione --ready e break quando arriva a 0

```
/* Altrimenti inserisce connsd tra i descrittori da controllare
   ed aggiorna maxd */
FD_SET(connsd, &allset);
if (connsd > maxd) maxd = connsd;
if (i > maxi) maxi = i;
if (--ready <= 0) /* Cicla finché ci sono ancora descrittori da controllare */
    continue;
}
/* Controlla i socket attivi per controllare se sono leggibili */
for (i = 0; i <= maxi; i++) {
    if ( (socksd = client[i]) < 0 )
        /* Se il descrittore non è stato selezionato, viene saltato */
        continue;
    if (FD_ISSET(socksd, &rset)) {
        /* Se socksd è leggibile, invoca la readline */
        if ((n = readline(socksd, buff, MAXLINE)) == 0) {
            /* Se legge EOF, chiude il descrittore di connessione */
            if (close(socksd) == -1) {
```

Multiplexing Server

Rimozione client: quando `recv()` dà 0 o errore $\rightarrow$ `close()`, `FD_CLR()`, `client[i] = -1`.

```
        perror("errore in close");
        exit(1);
    }
    /* Rimuove socksd dalla lista dei socket da controllare */
    FD_CLR(socksd, &allset);
    /* Cancella socksd da client */
    client[i] = -1;
}
else /* echo */
    if (writen(socksd, buff, n) < 0 ) {
        fprintf(stderr, "errore in write");
        exit(1);
    }
    if (--ready <= 0) break;
}
}
}
}
```

Multiplexing Server

Esercizio:

Si fornisce un echo server TCP già funzionante e mono-thread

Il server:

- accetta un singolo client alla volta
- ogni riga ricevuta dal client viene riinviata (echo)
- **ogni 10 secondi** invia anche una linea contenente il carattere x\n

Il client:

- legge da tastiera una riga, la invia al server, attende la risposta; riceve e mostra il messaggio periodico del server

Problema:

- Il client non è in grado di ricevere e mostrare i messaggi "x\n" inviati dal server, a meno che l'utente non inserisca nuovi comandi da tastiera

Modificare il client in modo che:

- utilizzi select() per gestire contemporaneamente:
 - l'input da **stdin**
 - i dati provenienti dal **socket**
- le righe digitate dall'utente siano inviate al server
- qualunque dato ricevuto dal server (echo oppure "x") sia stampato su **stdout**